

Course Introduction

Building Microservice Systems

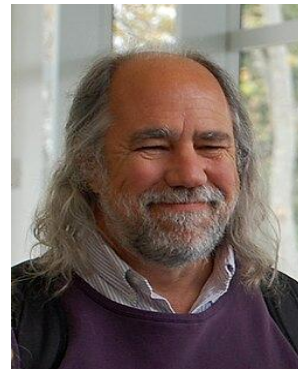
Ali Abrishami

Seyyed Hossein Mirhosseini



Software Architecture Definitions

- Architecture represents the **significant** design decisions that shape a system, where significant is measured by cost of change. (*“Grady Booch”*)
- If you think good architecture is expensive, try bad architecture. (*“Brian Foote”* and *“Joseph Yoder”*)
 - The authors of *Big Ball of Mud* paper



Grady Booch

Software Architecture Definitions (cont.)

- Architecture is the decisions that you wish you could get right **early** in a project, but that you are not necessarily more likely to get them right than any other. (*“Ralph Johnson” Co-author of Gang of Four book*)
- Architecture is a hypothesis, that needs to be proven by implementation and measurement. (*“Tom Gilb”*)
- **The only way to go fast, is to go well.** (*“Robert C. Martin”*)

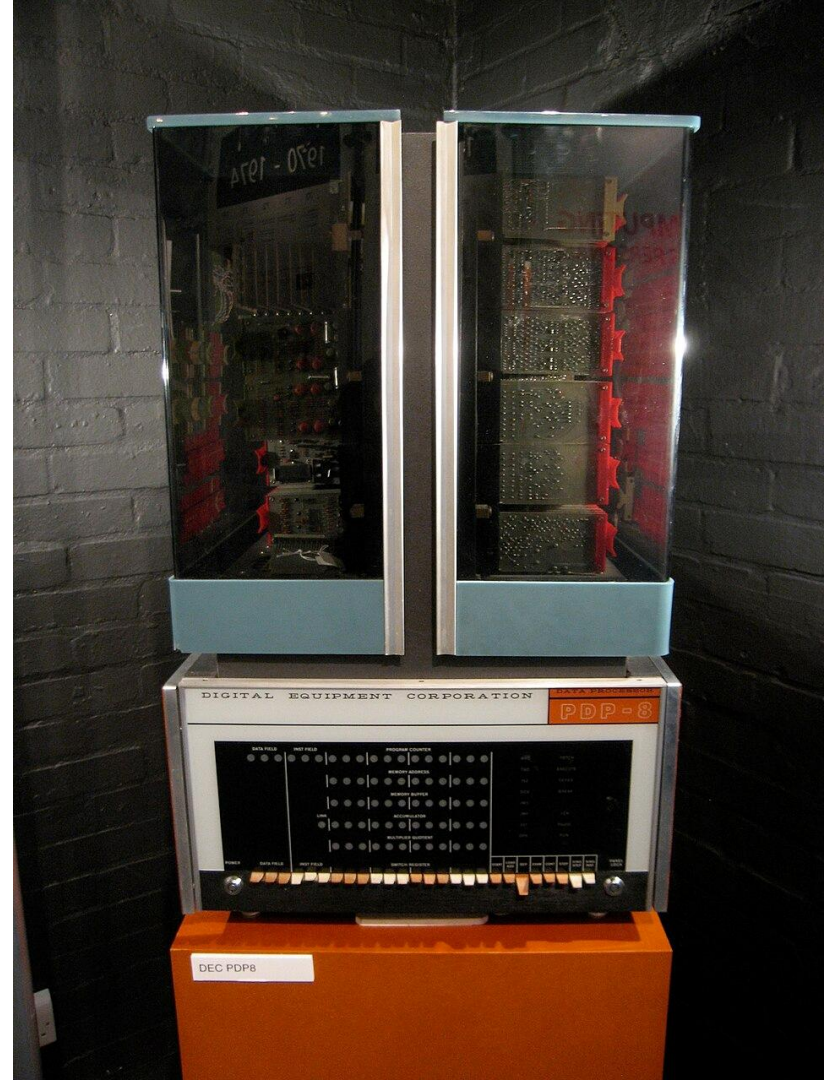


Robert C. Martin
(Uncle Bob)

Preface

Now, a program isn't really big until it exceeds 100,000 lines.

And if I transported you back to 1966 and showed you how to write and edit PDP-8 code by punching paper tape on a 10 character per second teletype, you might need 24 hours to recover from the disappointment.



Rules of Software Architecture

- But then you would be able to write the code. The code just hasn't changed that much.
- That's the secret: This changelessness of the code is the reason that the rules of software architecture are so consistent across system types.
- It doesn't take a huge amount of knowledge and skill to get a program working.
- It works because getting something to work—once—just isn't that hard.
- Getting it **right** is another matter entirely.

***The rules of software
architecture are
independent of every
other variable.***

The Result!

And when you get software right, something magical happens.

- You don't need:
 - hordes of programmers to keep it working.
 - You don't need massive requirements documents and huge issue tracking systems.
 - You don't need global cube farms and 24/7 programming.
-

And Other Results!

- When software is done right:
 - It requires a fraction of the human resources to create and maintain.
 - Changes are simple and rapid.
 - Defects are few and far between.
 - Effort is minimized, and functionality and flexibility are maximized.
-

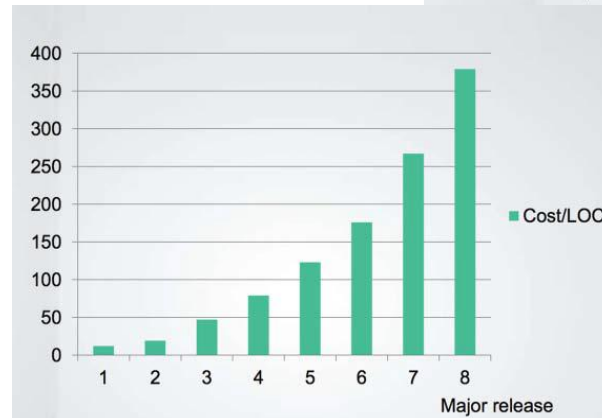
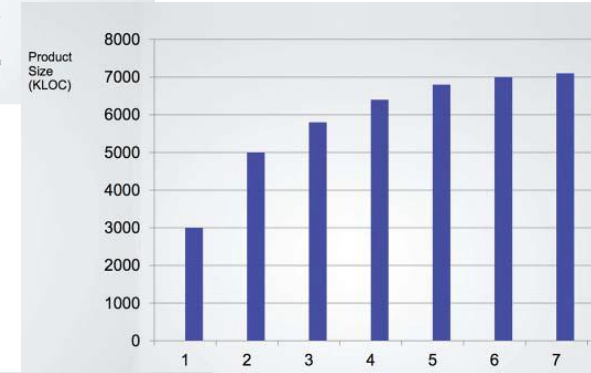
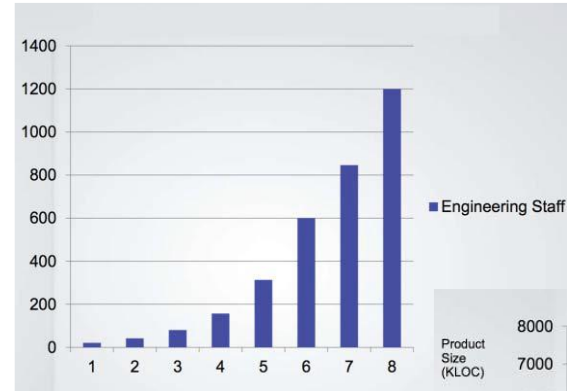
The problem or challenge



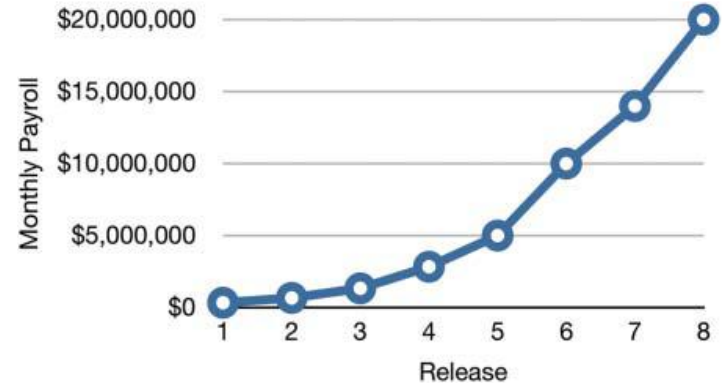
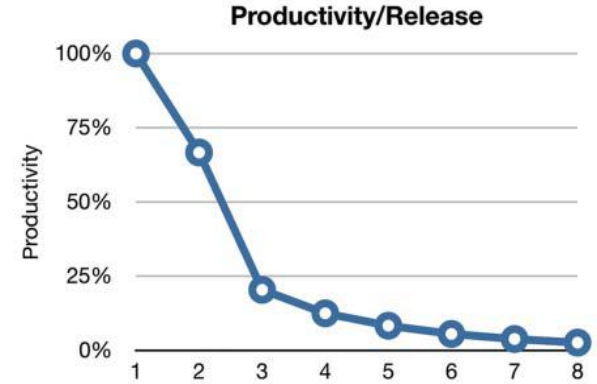
- Have you worked on systems that are so interconnected and intricately coupled that every change, regardless of how trivial, takes weeks and involves huge risks?

Growth of the engineering staff &
Productivity over the same period
of time & Cost per line of code
over time

And yet, despite all their heroics,
overtime, and dedication, they
simply aren't getting much of
anything done anymore.



Productivity by release & Monthly
development payroll by release



Tortoise and the Hare story:

“Slow and steady wins the race.”

*“The race is not to the swift, nor
the battle to the strong.”*

*“The more haste, the less
speed.”*



Conclusion

- The simple truth of software development:
The only way to go fast, is to go well.
- Start taking the **quality** of its software architecture seriously.
- To take software architecture seriously, you need to know what good software architecture is.
- To build a system with a design and an architecture that minimize effort and maximize productivity, you need to know which attributes of system architecture lead to that end.

Source: Clean Code

(Chapter 1)

This Course

Why This Course?

- Modern apps run across many servers and users worldwide
- They constantly evolve with new features and updates
- Poor architecture makes systems slow and fragile
- Monolithic systems are hard to scale and maintain
- Microservices split apps into small, independent services for flexibility

Course Learning Outcomes

After completing this course,
students will be able to:

- Analyze and evaluate software architectures
 - Apply clean architecture principles in real systems
 - Design systems using Domain-Driven Design
 - Build and reason about microservice architectures
 - Deploy and operate services using Docker & Kubernetes
-

How the Topics Fit Together

- Architecture principles → Foundation
- Clean Architecture → Internal structure
- DDD → Modeling complex business domains
- Microservices → System-level decomposition
- DevOps → Building & running architectures in production

Tools & Technologies

- Suggested Languages:
 - **Python**
 - **C#**
- Suggested Framework:
 - **FastAPI**
 - **DotNET**
- Deployment: **Docker & Kubernetes**
- **Git / GitHub**



kubernetes

Course Information

- Course title & code:
 - Advanced Topics in Computer Engineering (40222)
 - Building Microservice Systems
- Degree program / department:
 - Undergraduate / Computer Engineering Dept.
- Semester: Spring 2026

Contact Information

- Instructors' information:
 - Ali Abrishami
 - a.abrishami110@gmail.com
 - abrishami@ce.sharif.edu
 - Seyyed Hossein (Keivan) Mirhosseini
 - keivan.mirhosseini@gmail.com
 - hosein.mirhoseini77@sharif.edu
- Head TA: Mani Ebrahimi (mani.ebra@gmail.com)

Assessment overview

- Assignments: (6 points)
- Exams: Midterm & Final (6 points)
- Projects: Two phases (8 + 1 points)
- Extra points: Quizzes & Class Activity (1 point)

- Quera link
 - https://quera.org/course/add_to_course/course/27186/

Expectations from Students

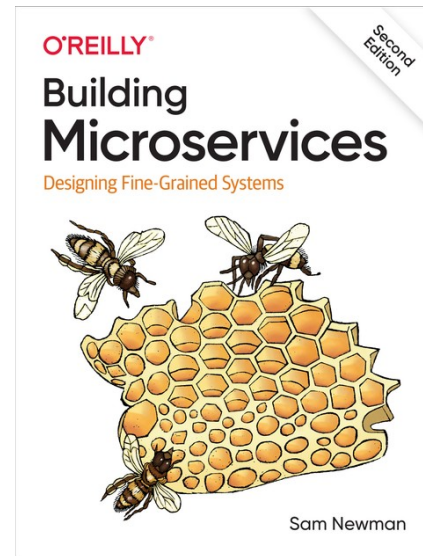
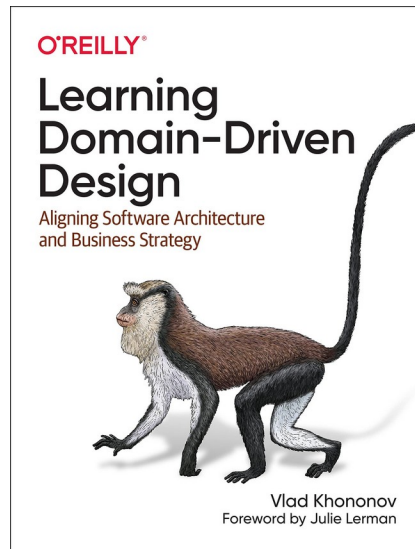
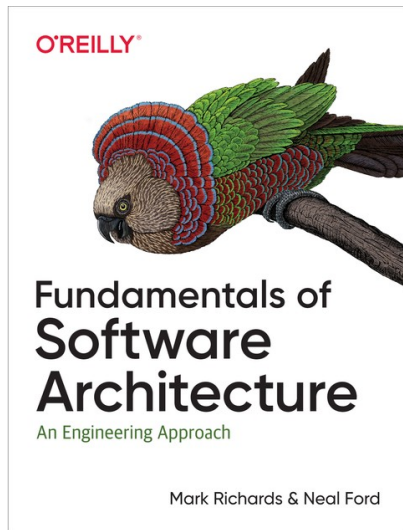
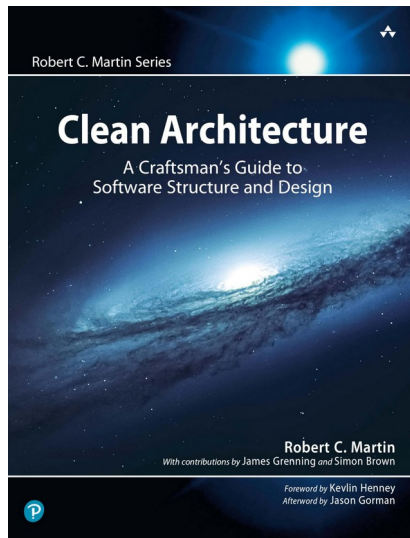
- Active participation in discussions
- Willingness to think beyond code
- Comfort with ambiguity and trade-offs
- Consistent work throughout the semester

Prerequisites / assumed knowledge

- Required skills or courses:
 - Web Programming Course
 - System Analysis & Design
- What students are expected to already know
- Prior industry work

Textbooks

- Sam Newman, “Building Microservices: Designing Fine-Grained Systems,” O’Reilly Media, 2015 (2nd Edition, 2021).
- Vlad Khononov, “Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy,” O’Reilly Media, 2021.
- Robert C. Martin, “Clean Architecture: A Craftsman’s Guide to Software Structure and Design,” Prentice Hall, 2017.
- Mark Richards and Neal Ford, “Fundamentals of Software Architecture: An Engineering Approach,” O’Reilly Media, 2020.



Read More

- <https://www.laputan.org/mud/> (Big Ball of Mud)

Q&A
